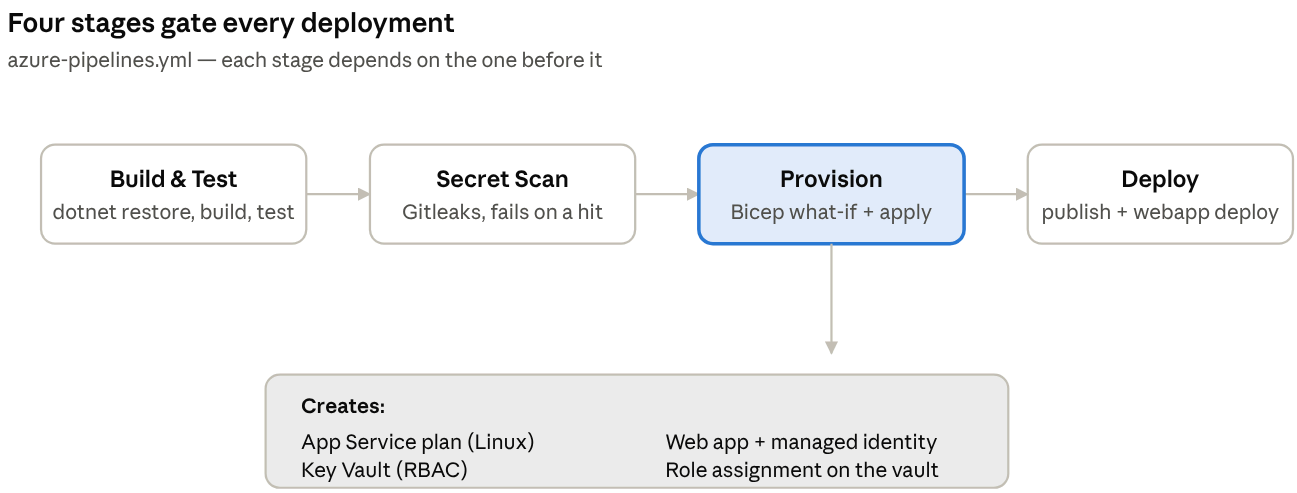
Secure Azure DevOps Pipeline: Bicep, Key Vault & Scanning Gates

Oct 29, 2026 · @Durai

A complete, self-contained walkthrough: provision a .NET 8 Web API's infrastructure from an Azure DevOps pipeline using Bicep, pull a secret from Key Vault through a managed identity with no credentials anywhere in code or YAML, and add two gates — secret scanning and dependency scanning — that fail the build on critical findings. Every code block and command below is copy-pasteable; the only things you need to substitute are your own subscription, resource group, and organization names.



pipeline flow · 4 stages, Provision creates 4 Azure resources

The case study in one picture: a secret-scan gate and a dependency-scan gate must pass before infrastructure is provisioned, and only a successful provision leads to a deploy. `Provision` is the stage that actually creates the four Azure resources shown below it — the App Service plan, the web app (with its managed identity), the Key Vault, and the role assignment that lets the web app read secrets from it.

Prerequisites

- An Azure subscription you can create resources in.
- Azure CLI installed and signed in.
- .NET 8 SDK installed (`dotnet --version` should report `8.0.x`).

- Git installed, with your identity configured (`git config --global user.name` / `user.email`).
- An Azure DevOps organization with a project you can create pipelines and service connections in.

Sign in and create a resource group dedicated to this walkthrough:

```
az login
az account set --subscription "<your subscription name or id>"
az group create --name rg-pharmacyapi-demo --location <your region, e.g.
eastus>
```

Create the Azure DevOps project

1. Go to `dev.azure.com` and sign in to your organization.
2. Click **New project** (top right).
3. **Project name:** `PharmacyApi-DevSecOps-Demo`.
4. **Visibility:** Private.
5. Expand **Advanced** → **Version control:** Git → **Work item process:** Basic (or your usual default).
6. Click **Create**.

Create the service connection

Azure DevOps needs its own identity to deploy into your subscription on the pipeline's behalf.

1. In the new project, click **Project settings** (bottom left gear icon).
2. Under **Pipelines**, click **Service connections**.
3. Click **New service connection**.
4. Select **Azure Resource Manager** → **Next**.
5. Select **Workload identity federation (automatic)** → **Next**.
6. **Scope level:** Resource Group.
7. **Subscription:** select your subscription.
8. **Resource group:** `rg-pharmacyapi-demo` (or whatever you created above).
9. **Service connection name:** `arm-pharmacyapi-demo-dev` — the YAML throughout this guide uses this exact name.

10. Check **Grant access permission to all pipelines**.
11. Click **Save**.

This connection is scoped to your resource group only — the pipeline can never reach anything else in your subscription.

Part 1 — Scaffold the .NET 8 API and the base pipeline

1. Create the solution (from wherever you keep your projects):

```
dotnet new webapi -n PharmacyApi.Demo --use-minimal-apis -o PharmacyApi.Demo
cd PharmacyApi.Demo
dotnet new xunit -n PharmacyApi.Demo.Tests -o ../PharmacyApi.Demo.Tests
cd ..
dotnet new sln -n PharmacyApi.Demo
dotnet sln add PharmacyApi.Demo/PharmacyApi.Demo.csproj
dotnet sln add PharmacyApi.Demo.Tests/PharmacyApi.Demo.Tests.csproj
```

2. Replace `PharmacyApi.Demo/Program.cs` with a minimal API:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/prescriptions/{id}", (int id) =>
{
    return Results.Ok(new
    {
        Id = id,
        DrugName = "Amoxicillin",
        Dosage = "500mg"
    });
});

app.MapGet("/health", () => Results.Ok("healthy"));

app.Run();
```

3. Add a `Dockerfile` at the solution root:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish PharmacyApi.Demo/PharmacyApi.Demo.csproj -c Release -o
/app

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
COPY --from=build /app .
ENTRYPOINT ["dotnet", "PharmacyApi.Demo.dll"]
```

4. Add a `.gitignore`, then push to Azure Repos:

```
dotnet new gitignore
git init
git add .
git commit -m "Initial PharmacyApi.Demo scaffold"
git remote add origin https://dev.azure.com/<your-org>/PharmacyApi-DevSecOps-
Demo/_git/PharmacyApi-DevSecOps-Demo
git push -u origin main
```

5. Add the base pipeline as `azure-pipelines.yml` at the repo root, then commit and push it too:

```
trigger:
  - main

pool:
  vmImage: 'ubuntu-latest'

stages:
- stage: BuildAndTest
  jobs:
  - job: Build
    steps:
    - task: UseDotNet@2
      inputs:
        version: '8.0.x'
    - script: dotnet restore
      displayName: 'Restore'
```

```
    - script: dotnet build --configuration Release --no-restore
      displayName: 'Build'
    - script: dotnet test --configuration Release --no-build
      displayName: 'Test'
```

6. Wire up the pipeline in Azure DevOps:

1. Open **Pipelines → New pipeline**.
2. Select **Azure Repos Git** → select your repo.
3. Select **Existing Azure Pipelines YAML file**.
4. **Branch:** main, **Path:** `/azure-pipelines.yml` → **Continue**.
5. Click **Run**.

Confirm the `BuildAndTest` stage goes green before moving on.

Part 2 — Provision infrastructure with Bicep from the pipeline

1. Create `infra/main.bicep`:

```
param environmentName string
param location string = resourceGroup().location

resource appPlan 'Microsoft.Web/serverfarms@2023-01-01' = {
  name: 'asp-pharmacyapi-${environmentName}'
  location: location
  kind: 'linux'
  sku: {
    name: 'B1'
    tier: 'Basic'
  }
  properties: {
    reserved: true // required for a Linux plan — omitting it provisions
Windows
  }
}

resource webApp 'Microsoft.Web/sites@2023-01-01' = {
  name: 'app-pharmacyapi-${environmentName}'
  location: location
  kind: 'app,linux'
  properties: {
    serverFarmId: appPlan.id
```

```
    siteConfig: {
      linuxFxVersion: 'DOTNETCORE|8.0'
    }
  }
}
```

2. Create `infra/dev.bicepparam`:

```
using 'main.bicep'

param environmentName = 'demo-dev'
```

3. Add a `Provision` stage to `azure-pipelines.yml`, right after `BuildAndTest`. Run `what-if` first so you can preview the diff before anything actually changes:

```
- stage: Provision
  dependsOn: BuildAndTest
  jobs:
  - job: ProvisionInfrastructure
    steps:
    - task: AzureCLI@2
      displayName: 'What-if preview'
      inputs:
        azureSubscription: 'arm-pharmacyapi-demo-dev'
        scriptType: 'bash'
        scriptLocation: 'inlineScript'
        inlineScript: |
          az deployment group what-if \
            --resource-group rg-pharmacyapi-demo \
            --template-file infra/main.bicep \
            --parameters infra/dev.bicepparam
    - task: AzureCLI@2
      displayName: 'Apply'
      inputs:
        azureSubscription: 'arm-pharmacyapi-demo-dev'
        scriptType: 'bash'
        scriptLocation: 'inlineScript'
        inlineScript: |
          az deployment group create \
            --resource-group rg-pharmacyapi-demo \
            --template-file infra/main.bicep \
            --parameters infra/dev.bicepparam
```

4. Commit, push. In the pipeline run, the **What-if preview** step lists what will be created; the **Apply** step actually creates it.

5. Verify in the Portal: Resource groups → open `rg-pharmacyapi-demo` → confirm `asp-pharmacyapi-demo-dev` (App Service Plan) and `app-pharmacyapi-demo-dev` (Web App) are listed, both Linux.

6. Prove it's idempotent — change `main.bicep`'s plan `sku` from `B1` / `Basic` to `S1` / `Standard`, commit, push. The **What-if preview** step should show only that one property changing (a `~` modify), not a full recreate — proof the deployment only touches what actually changed.

Part 3 — Key Vault and a managed identity

The moment anything needs a real credential (a connection string, an API key), it should never live in `appsettings.json`, a pipeline variable in plain text, or anywhere hardcoded. This part gives the web app its own identity in Azure AD and grants that identity — and only that identity — read access to one Key Vault.

1. Extend `infra/main.bicep` — replace the `webApp` resource with this identity-enabled version, and add `keyVault` and `secretsUserRole` below it. `appPlan` stays unchanged from Part 2:

```
resource webApp 'Microsoft.Web/sites@2023-01-01' = {
  name: 'app-pharmacyapi-${environmentName}'
  location: location
  kind: 'app,linux'
  identity: {
    type: 'SystemAssigned'
  }
  properties: {
    serverFarmId: appPlan.id
    siteConfig: {
      linuxFxVersion: 'DOTNETCORE|8.0'
      appSettings: [
        {
          name: 'KeyVaultUri'
          value: keyVault.properties.vaultUri
        }
      ]
    }
  }
```

```
}

resource keyVault 'Microsoft.KeyVault/vaults@2023-07-01' = {
  name: 'kv-pharmacyapi-${environmentName}'
  location: location
  properties: {
    sku: {
      family: 'A'
      name: 'standard'
    }
    tenantId: subscription().tenantId
    enableRbacAuthorization: true
  }
}

resource secretsUserRole 'Microsoft.Authorization/roleAssignments@2022-04-01'
= {
  name: guid(keyVault.id, webApp.id, 'KeyVaultSecretsUser')
  scope: keyVault
  properties: {
    roleDefinitionId: subscriptionResourceId(
      'Microsoft.Authorization/roleDefinitions',
      '4633458b-17de-408a-b874-0445c86b69e6' // Key Vault Secrets User
    )
    principalId: webApp.identity.principalId
    principalType: 'ServicePrincipal'
  }
}
```

> **Grant your pipeline's identity permission to assign roles, before you push.** Creating a role assignment is a privileged action — `Microsoft.Authorization/roleAssignments/write` — that Contributor doesn't include. Run this once, as a subscription or resource-group owner:
>
> ```
> az role assignment create \
>   --assignee-object-id <service connection's SP object ID> \
>   --assignee-principal-type ServicePrincipal \
>   --role "User Access Administrator" \
>   --scope "/subscriptions/<sub-id>/resourceGroups/rg-pharmacyapi-demo"
> ```
>
> Find the service connection's object ID in Azure DevOps under **Project settings → Service connections → (your connection) → Manage Service Principal**, or from its App Registration in Entra ID. Scope it to the resource group, not narrower — the Key Vault doesn't exist yet at this point, so a vault-scoped grant isn't possible.

2. Commit, push. Confirm the Provision stage creates the Key Vault and the role assignment. Then verify: Portal → the Key Vault → **Access control (IAM)** → **Role assignments** tab → confirm **Key Vault Secrets User** is listed, assigned to `app-pharmacyapi-demo-dev`.

3. Add a secret (a stand-in for a real database connection string): Portal → the Key Vault → **Objects → Secrets → + Generate/Import**:

- Name: `SqlConnectionString`
- Value: `Server=tcp:demo.database.windows.net;Database=PharmacyDb;User=demo;Password=demo123!`
- Click **Create**.

> If you get "unauthorized" trying to add the secret yourself, see **Common Issues** below — you need a data-plane role on the vault too, separate from the web app's role.

4. Update `PharmacyApi.Demo/Program.cs` to read the secret through the managed identity — no connection string anywhere in code, config, or YAML:

```
using Azure.Identity;

var builder = WebApplication.CreateBuilder(args);

var keyVaultUri = builder.Configuration["KeyVaultUri"];
if (!string.IsNullOrEmpty(keyVaultUri))
{
    builder.Configuration.AddAzureKeyVault(
        new Uri(keyVaultUri),
        new DefaultAzureCredential());
}

var app = builder.Build();

app.MapGet("/health", (IConfiguration config) =>
{
    var hasSecret = !string.IsNullOrEmpty(config["SqlConnectionString"]);
    return Results.Ok(new { status = "healthy", secretLoaded = hasSecret });
});

app.MapGet("/prescriptions/{id}", (int id) =>
    Results.Ok(new { Id = id, DrugName = "Amoxicillin", Dosage = "500mg" }));

app.Run();
```

Add the two NuGet packages this needs:

```
dotnet add PharmacyApi.Demo package Azure.Identity
dotnet add PharmacyApi.Demo package
Azure.Extensions.AspNetCore.Configuration.Secrets
```

`KeyVaultUri` is the only thing in app settings — and it isn't a secret, just a pointer. `DefaultAzureCredential` picks up the web app's managed identity automatically at runtime; locally (outside Azure) it falls back to your own `az login` session.

5. Add a `Deploy` stage after `Provision`:

```
- stage: Deploy
  dependsOn: Provision
  jobs:
  - job: DeployApp
    steps:
    - task: UseDotNet@2
      inputs:
        version: '8.0.x'
    - script: dotnet publish PharmacyApi.Demo/PharmacyApi.Demo.csproj -c
Release -o $(Build.ArtifactStagingDirectory)
      displayName: 'Publish'
    - task: AzureWebApp@1
      inputs:
        azureSubscription: 'arm-pharmacyapi-demo-dev'
        appName: 'app-pharmacyapi-demo-dev'
        package: '$(Build.ArtifactStagingDirectory)'
```

6. Commit and push the `.csproj` and `Program.cs` changes together — a `PackageReference` added without its matching code change (or vice versa) won't build. Once the pipeline is green:

- Portal → the Web App → copy its URL from **Overview**.
- Open `<url>/health` — confirm `"secretLoaded": true`.
- Portal → the Web App → **Monitoring → Log stream** → show it running, and that the connection string never appears anywhere: not in logs, not in the YAML, not in the source.

Part 4 — Secret-scanning gate

1. Add a `SecretScan` stage, right after `BuildAndTest` and before `Provision`:

```
- stage: SecretScan
  dependsOn: BuildAndTest
  jobs:
  - job: ScanForSecrets
    steps:
    - script: |
        docker run --rm -v $(Build.SourcesDirectory):/repo \
          zricethezav/gitleaks:latest detect \
          --source="/repo" --no-git -v --exit-code 1
      displayName: 'Gitleaks secret scan'
```

Update `Provision`'s `dependsOn` from `BuildAndTest` to `SecretScan`, so provisioning only runs once the scan passes.

2. Commit, push. Confirm `SecretScan` passes clean.

3. Plant a fake secret to prove the gate works — add this as a comment anywhere in `Program.cs`:

```
// TEMP_AWS_KEY = "AKIAZQ3DRRMXXW2VKPQF"
```

> Don't use AWS's own documented example key (`AKIAIOSFODNN7EXAMPLE`) for this. Gitleaks' built-in AWS rule applies a minimum-entropy filter specifically so it doesn't flag that exact string, since it appears in countless public tutorials and READMEs. Use a random-looking value like the one above — it still matches the AWS key shape (`AKIA` + 16 characters), but has real entropy, so it actually trips the rule.

4. Commit, push. `SecretScan` should fail red — Gitleaks finds the key, reports its rule id (`aws-access-token`) and the file/line, and exits non-zero, blocking `Provision` from ever running.

5. Fix it — remove the line, commit again, confirm green. Worth stating explicitly: that first commit with the fake key is still sitting in the repository's git history. Deleting the line isn't the fix for a real leak — rotating the actual credential at its source is.

Part 5 — Dependency vulnerability scanning gate

The .NET 8 SDK has NuGet Audit built in — no extra tool to install.

1. Turn on the audit as a hard gate. Add this inside `PharmacyApi.Demo.csproj`'s `<PropertyGroup>`:

```
<NuGetAuditMode>all</NuGetAuditMode>
<NuGetAuditLevel>moderate</NuGetAuditLevel>
<WarningsAsErrors>NU1902;NU1903;NU1904</WarningsAsErrors>
```

`NU1902` = Moderate, `NU1903` = High, `NU1904` = Critical — this turns those audit warnings into build failures.

2. Commit, push. Confirm the build still passes clean.

3. Add a known-vulnerable package. Advisory databases change as packages get patched, so confirm a current example before your session:

```
dotnet list package --vulnerable --include-transitive
```

A long-standing, commonly-cited example to start from if nothing else turns up:

```
dotnet add PharmacyApi.Demo package Newtonsoft.Json --version 9.0.1
```

4. Commit, push. Open the pipeline run and watch `dotnet restore` fail inside `BuildAndTest`, with the CVE id printed directly in the log (`NU1903` , the advisory URL, the package and version).

5. Fix it — remove the vulnerable package reference (or bump it to a patched version), commit again, confirm the build goes green.

Full pipeline, assembled

Everything from Parts 1–5 together — `BuildAndTest` runs first; `SecretScan` and `Provision` both depend on it, with `Provision` additionally gated behind `SecretScan` passing; `Deploy` runs last, after `Provision` succeeds:

```
trigger:
  - main

pool:
  vmImage: 'ubuntu-latest'

stages:
- stage: BuildAndTest
  jobs:
  - job: Build
    steps:
```

```
    - task: UseDotNet@2
      inputs:
        version: '8.0.x'
    - script: dotnet restore
      displayName: 'Restore (runs NuGet Audit)'
    - script: dotnet build --configuration Release --no-restore
      displayName: 'Build'
    - script: dotnet test --configuration Release --no-build
      displayName: 'Test'

- stage: SecretScan
  dependsOn: BuildAndTest
  jobs:
  - job: ScanForSecrets
    steps:
    - script: |
        docker run --rm -v $(Build.SourcesDirectory):/repo \
          zricethezav/gitleaks:latest detect \
          --source="/repo" --no-git -v --exit-code 1
      displayName: 'Gitleaks secret scan'

- stage: Provision
  dependsOn: SecretScan
  jobs:
  - job: ProvisionInfrastructure
    steps:
    - task: AzureCLI@2
      displayName: 'What-if preview'
      inputs:
        azureSubscription: 'arm-pharmacyapi-demo-dev'
        scriptType: 'bash'
        scriptLocation: 'inlineScript'
        inlineScript: |
          az deployment group what-if \
            --resource-group rg-pharmacyapi-demo \
            --template-file infra/main.bicep \
            --parameters infra/dev.bicepparam
    - task: AzureCLI@2
      displayName: 'Apply'
      inputs:
        azureSubscription: 'arm-pharmacyapi-demo-dev'
        scriptType: 'bash'
        scriptLocation: 'inlineScript'
```

```
        inlineScript: |
          az deployment group create \
            --resource-group rg-pharmacyapi-demo \
            --template-file infra/main.bicep \
            --parameters infra/dev.bicepparam

- stage: Deploy
  dependsOn: Provision
  jobs:
  - job: DeployApp
    steps:
    - task: UseDotNet@2
      inputs:
        version: '8.0.x'
    - script: dotnet publish PharmacyApi.Demo/PharmacyApi.Demo.csproj -c
Release -o $(Build.ArtifactStagingDirectory)
      displayName: 'Publish'
    - task: AzureWebApp@1
      inputs:
        azureSubscription: 'arm-pharmacyapi-demo-dev'
        appName: 'app-pharmacyapi-demo-dev'
        package: '$(Build.ArtifactStagingDirectory)'
```

And the complete `infra/main.bicep` it references:

```
param environmentName string
param location string = resourceGroup().location

resource appPlan 'Microsoft.Web/serverfarms@2023-01-01' = {
  name: 'asp-pharmacyapi-${environmentName}'
  location: location
  kind: 'linux'
  sku: {
    name: 'S1'
    tier: 'Standard'
  }
  properties: {
    reserved: true
  }
}

resource webApp 'Microsoft.Web/sites@2023-01-01' = {
  name: 'app-pharmacyapi-${environmentName}'
```

```
  location: location
  kind: 'app,linux'
  identity: {
    type: 'SystemAssigned'
  }
  properties: {
    serverFarmId: appPlan.id
    siteConfig: {
      linuxFxVersion: 'DOTNETCORE|8.0'
      appSettings: [
        {
          name: 'KeyVaultUri'
          value: keyVault.properties.vaultUri
        }
      ]
    }
  }
}

resource keyVault 'Microsoft.KeyVault/vaults@2023-07-01' = {
  name: 'kv-pharmacyapi-${environmentName}'
  location: location
  properties: {
    sku: {
      family: 'A'
      name: 'standard'
    }
    tenantId: subscription().tenantId
    enableRbacAuthorization: true
  }
}

resource secretsUserRole 'Microsoft.Authorization/roleAssignments@2022-04-01'
= {
  name: guid(keyVault.id, webApp.id, 'KeyVaultSecretsUser')
  scope: keyVault
  properties: {
    roleDefinitionId: subscriptionResourceId(
      'Microsoft.Authorization/roleDefinitions',
      '4633458b-17de-408a-b874-0445c86b69e6'
    )
```

```
    principalId: webApp.identity.principalId
    principalType: 'ServicePrincipal'
  }
}
```

And `PharmacyApi.Demo.csproj` in its final state:

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <NuGetAuditMode>all</NuGetAuditMode>
    <NuGetAuditLevel>moderate</NuGetAuditLevel>
    <WarningsAsErrors>NU1902;NU1903;NU1904</WarningsAsErrors>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.33"
/>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
    <PackageReference Include="Azure.Identity" Version="1.21.0" />
    <PackageReference
Include="Azure.Extensions.AspNetCore.Configuration.Secrets" Version="1.5.2"
/>
  </ItemGroup>

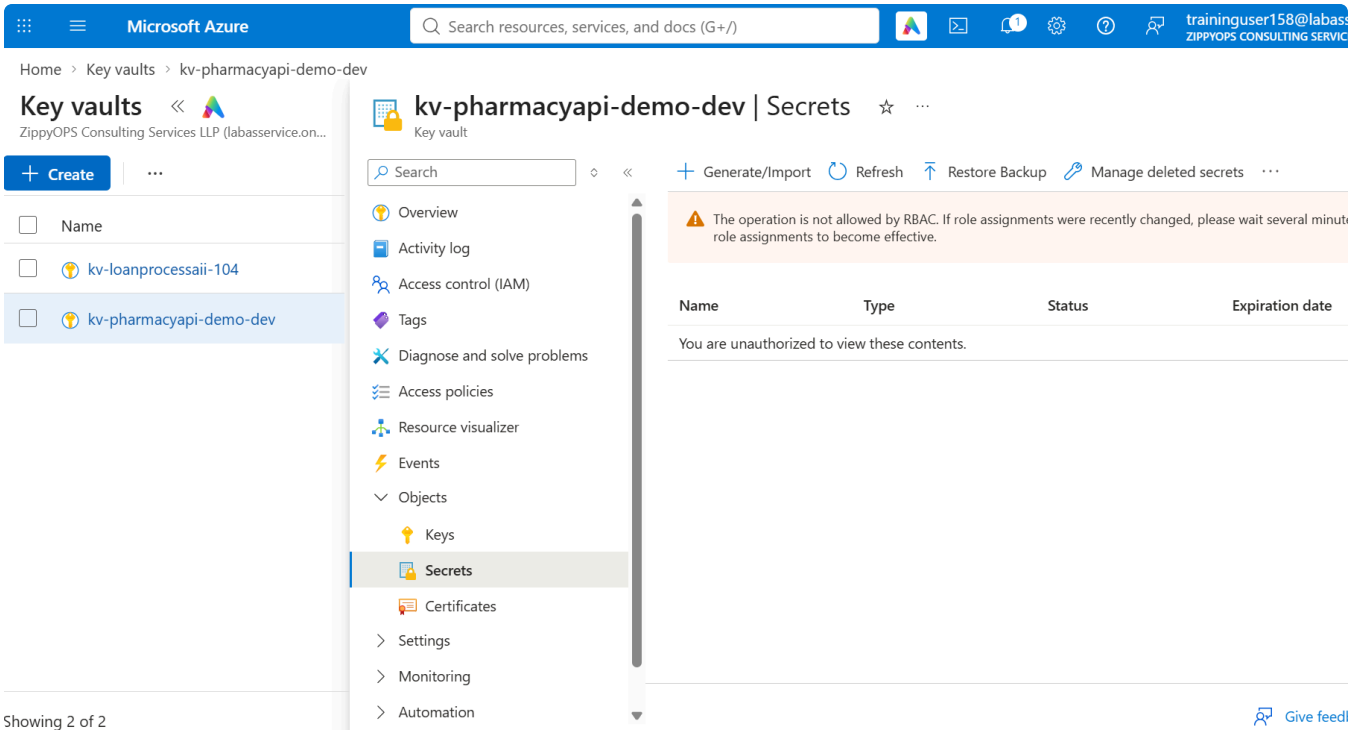
</Project>
```

Common issues

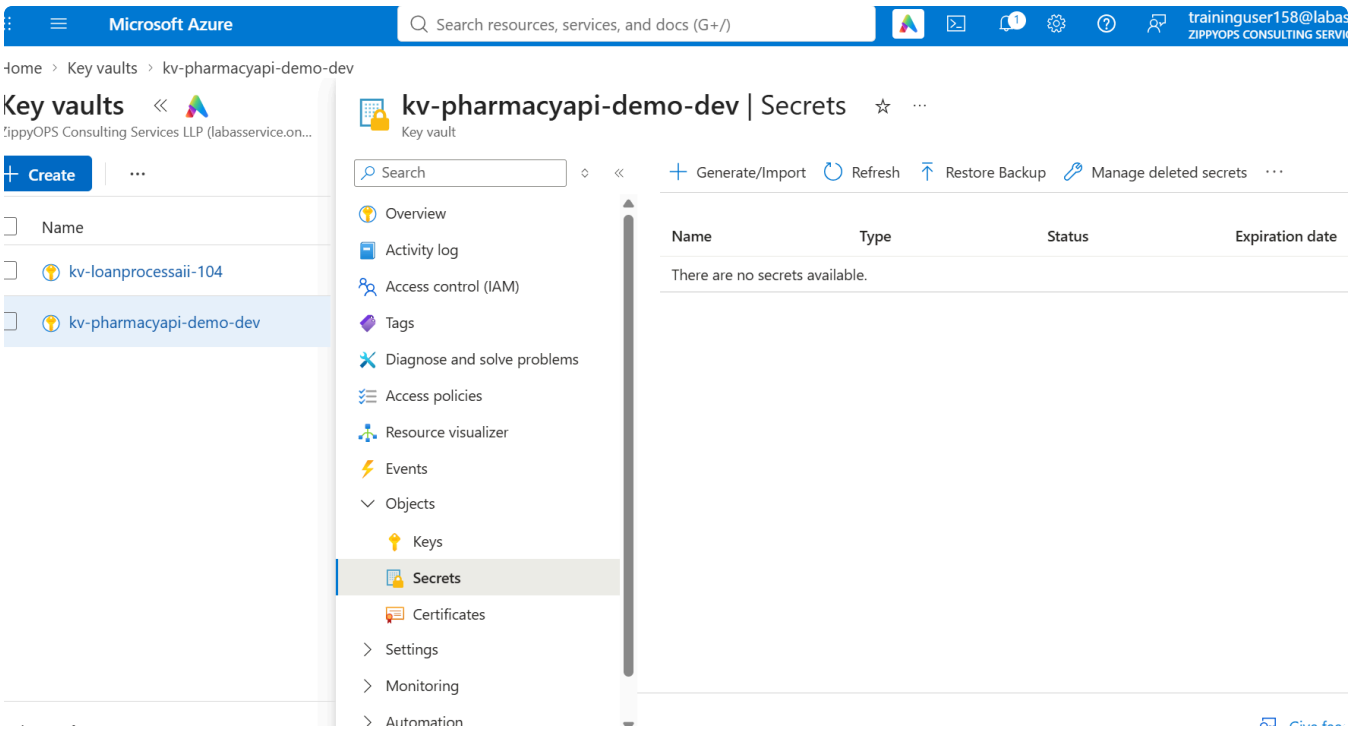
Everything below actually happened while building this walkthrough — none of it is hypothetical.

Symptom	Cause	Fix
Bicep deploy succeeds but the App Service plan is still Windows	A previously failed deployment left a Windows plan in place; a Linux-only `main.bicep` without explicit `kind` and `reserved` properties doesn't force a switch	Set `kind: 'linux'` and `properties.reserved: true` on the plan, and `kind: 'app,linux'` on the web app

Symptom	Cause	Fix
`InvalidTemplateDeployment` — `Authorization failed... Microsoft.Authorization/roleAssignments/write`	The pipeline's service connection only has **Contributor**, which deliberately excludes creating role assignments	Grant the service connection's service principal **User Access Administrator** at the resource-group scope: `az role assignment create --assignee-object-id <sp-object-id> --assignee-principal-type ServicePrincipal --role "User Access Administrator" --scope /subscriptions/<sub-id>/resourceGroups/<rg-name>`
Portal says "The operation is not allowed by RBAC" when you try to add a secret by hand	`enableRbacAuthorization: true` on the vault means even your own human sign-in needs an explicit RBAC role — access policies don't apply anymore	`az role assignment create --assignee <your-email> --role "Key Vault Secrets Officer" --scope <vault-resource-id>`, then wait — see next row



Symptom	Cause	Fix
Role assignment created successfully but access still fails for a few minutes	RBAC role assignments take a few minutes to propagate through Azure — this applies to service principals and human accounts alike	Just wait and retry; there's nothing wrong with the assignment itself



Symptom	Cause	Fix
Gitleaks reports "no leaks found" even though you planted an AWS-looking key	Gitleaks' built-in AWS rule applies an entropy filter (~3.0 threshold), and it specifically excludes AWS's own well-known documentation example key `AKIAIOSFODNN7EXAMPLE`	Use a higher-entropy fake key instead, e.g. `AKIAZQ3DRRMXXW2VKPQF` — it will trigger `RuleID: aws-access-token` and exit code 1
Build fails with `CS0246: The type or namespace name 'Azure' could not be found`	`dotnet add package` was run locally, but the resulting `.csproj` change was never committed and pushed — the pipeline builds from Azure Repos, not your local disk	Check `git status` locally; if the `.csproj` shows as modified, `git add`, `git commit`, `git push`
Pipeline restore shows `NU1504` duplicate `PackageReference` warnings	The same package was added once locally and once directly in the Azure Repos web editor, producing two `<PackageReference>` entries for it	Pick one editing location going forward; `git pull` before editing locally to stay in sync, and clean up the `.csproj` to one entry per package at the newer version

Avoiding the local-vs-web-editor sync trap

If you edit in both your local clone and the Azure Repos web editor without syncing between them, you'll eventually get conflicting or duplicated changes (as in the `NU1504` row above). Pick one of these habits:

- **Local-only:** always `git pull` before you start editing, and `git push` as soon as you're done.
- **Web-editor-only:** avoid touching the local clone at all for that file.
- **Switching between the two:** always `git pull` immediately before switching, so you start from what's actually on the server.

Cleanup

If you created a dedicated resource group just for this walkthrough, the simplest teardown is deleting the whole thing:

```
az group delete --name rg-pharmacyapi-demo --yes --no-wait
```

If you provisioned into a resource group you share with other work, delete just what this walkthrough created instead:

```
az webapp delete --name app-pharmacyapi-demo-dev --resource-group rg-
pharmacyapi-demo
az appservice plan delete --name asp-pharmacyapi-demo-dev --resource-group
rg-pharmacyapi-demo --yes
az keyvault delete --name kv-pharmacyapi-demo-dev --resource-group rg-
pharmacyapi-demo
az keyvault purge --name kv-pharmacyapi-demo-dev
```

Also remove the role assignment you created by hand in Part 3 — it doesn't get cleaned up automatically:

```
az role assignment list \
  --assignee <service connection's object ID> \
  --scope "/subscriptions/<sub-id>/resourceGroups/rg-pharmacyapi-demo" -o
table

az role assignment delete \
  --assignee <service connection's object ID> \
  --role "User Access Administrator" \
  --scope "/subscriptions/<sub-id>/resourceGroups/rg-pharmacyapi-demo"
```

Finally, decide whether to keep or delete the Azure DevOps project (`PharmacyApi-DevSecOps-Demo`) and its repo, depending on whether you'll run this walkthrough again.